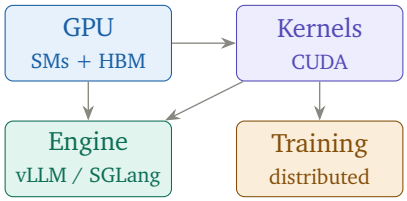
BOOK 2

LLM Systems Engineering

CUDA, distributed training, and inference engine internals



The companion volume to
LLM Inference, Sampling, and Context Engineering

Carmel Prosper Sagbo

Research Associate & ML Infrastructure Engineer
CyLab-Africa / Upanzi Network
Carnegie Mellon University Africa, Kigali

June 2026

For internal study and reference

Contents

About This Volume

Book 1, *LLM Inference, Sampling, and Context Engineering*, answered the question *how do I use and serve a model well?* It treated the GPU, the kernels, and the training process as boxes that simply work. This volume opens those boxes.

> **In plain terms** If Book 1 taught you to drive the car and read the dashboard, Book 2 takes you under the hood: the engine, the transmission, the fuel system. You will learn what a GPU actually is, how a CUDA kernel runs on it, how a model is trained across hundreds of those GPUs, and how a production inference engine like vLLM is built internally.

This is the harder volume. It assumes you have read Book 1, or are comfortable with attention, the prefill/decode split, the KV cache, and the memory-bandwidth bottleneck. Where a concept from Book 1 is load-bearing, the text says so explicitly, so you know when to glance back.

The progression mirrors Book 1's philosophy — a diagram before every mechanism, a plain-language box for every hard idea, and hands-on labs at the end — but the subject is the systems layer beneath the application: hardware, kernels, distributed training, and engine internals. The goal is to move you from *understanding* inference systems to being able to *design and optimize* them.

It also folds in four topics that a pure application volume could only gesture at: a real GPU-architecture mental model, where training sits relative to inference, full agent system architecture, and the cost modeling that turns latency numbers into dollars.

PART I

Hardware Foundations

Chapter 1

GPU Architecture for LLM Inference

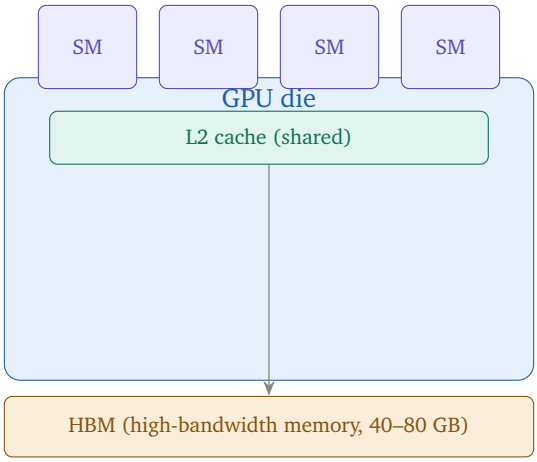
Throughout Book 1 you met H100, A100, HBM, tensor cores, and SRAM as names. This chapter gives them a structure. You do not need to design chips, but you need a mental model accurate enough that Flash Attention, quantization, and tensor parallelism stop being incantations.

1.1 Why GPUs, Not CPUs

A CPU has a few powerful cores optimized for latency — finishing one complex task quickly. A GPU has thousands of simple cores optimized for throughput — doing the same simple operation on enormous amounts of data at once. Matrix multiplication, the core of every transformer layer, is exactly that: the same multiply-accumulate repeated across millions of numbers.

> **In plain terms** A CPU is a handful of professors, each solving a hard problem alone. A GPU is a stadium of students, each doing one small sum, all at the same instant. For multiplying big matrices, the stadium wins overwhelmingly.

1.2 The Anatomy of a GPU



The pieces, from smallest to largest:

- **SM (Streaming Multiprocessor)**: the GPU's unit of work. An H100 has 132 of them. Each SM contains many small arithmetic cores plus specialized tensor cores, and its own slice of fast on-chip memory.

- **Tensor cores**: dedicated matrix-multiply units inside each SM. They perform a small matrix multiply-accumulate in one operation, and they are the reason a GPU hits thousands of TFLOPS. Mixed-precision training and quantized inference exist largely to feed these cores in the formats they run fastest.
- **SRAM (shared memory)**: tiny (kilobytes per SM), extremely fast on-chip memory. This is the scratchpad Flash Attention keeps the attention tiles in to avoid touching HBM.
- **L2 cache**: a megabytes-scale cache shared across all SMs.
- **HBM (High-Bandwidth Memory)**: the large (40–80 GB) but comparatively slow DRAM where model weights and the KV cache live. The 3.35 TB/s figure from Book 1 is HBM bandwidth.

> The memory hierarchy is a pyramid: tiny-and-instant SRAM at the top, huge-and-slow HBM at the bottom. Every inference optimization in Book 1 is a move up this pyramid — keep data in SRAM, avoid round-trips to HBM. Flash Attention is the purest example; quantization is another (smaller data moves through the pyramid faster).

1.3 Connecting GPUs

One GPU is rarely enough for a large model. How GPUs talk to each other determines what kinds of parallelism are practical.

Link	Role
PCIe	GPU-to-CPU and GPU-to-GPU across the motherboard. Convenient, relatively slow (∼64 GB/s).
NVLink	Direct GPU-to-GPU, far faster (∼900 GB/s on H100). The fabric that makes tensor parallelism viable within a node.
InfiniBand	Node-to-node across a cluster. The fabric for multi-node training.

> Interconnect speed is why tensor parallelism (which needs constant fast GPU-to-GPU chatter) stays *within* an NVLink-connected node, while pipeline and data parallelism (which communicate less often) can span nodes over InfiniBand. The parallelism strategy is dictated by the wiring, not just the math.

1.4 Reading a Spec Sheet

GPU	HBM	Bandwidth	Typical use
L40S	48 GB	0.86 TB/s	cost-efficient inference
A100	80 GB	2.0 TB/s	training and inference workhorse
H100	80 GB	3.35 TB/s	frontier training and inference

When you read a spec sheet, the two numbers that predict LLM behaviour most are HBM capacity (does the model plus KV cache fit?) and HBM bandwidth (how fast does decode run?). Compute TFLOPS matters mostly for prefill and training.

Chapter 2

The CUDA Programming Model

CUDA is how you tell those thousands of cores what to do. You do not need to become a CUDA engineer to work on LLM systems, but you need to understand the model well enough to read a kernel and reason about why one is faster than another.

2.1 Threads, Blocks, Grids

A CUDA program launches a **kernel**: one function that runs simultaneously across many threads. Threads are organized in a hierarchy that mirrors the hardware.

- **Thread**: runs the kernel on one piece of data.
- **Block**: a group of threads that run on a single SM and can share that SM's fast SRAM and synchronize with each other. This is the key unit — threads in a block cooperate; threads in different blocks generally cannot.
- **Grid**: all the blocks launched for one kernel.

> **In plain terms** Think of a huge spreadsheet to fill in. Each cell is a thread. A block is a rectangular region worked by one team that shares a fast whiteboard (SRAM). The grid is the whole spreadsheet. You write one formula (the kernel); the GPU runs it in every cell at once.

2.2 The Warp

Hardware executes threads in groups of 32 called **warps**. All 32 threads in a warp run the same instruction at the same time (single-instruction, multiple-thread). This has a sharp consequence:

> **Warp divergence**: if threads in a warp take different branches of an `if`, the warp runs both branches serially with half the threads idle each time. Branchy code wastes the GPU. High-performance kernels are written to keep all 32 threads on the same path.

2.3 The Memory Model in Code

Writing a fast kernel is mostly about *where* data lives:

Memory	Scope and speed
Registers	per-thread, fastest. Hold the values a thread is actively computing on.
Shared (SRAM)	per-block, very fast. Threads in a block cooperate through it.
Global (HBM)	whole grid, slow. Where inputs and outputs live.

The entire craft of GPU kernel optimization reduces to one sentence: **move data from global memory into shared memory and registers as few times as possible, and do as much work as you can while it is there.** This is exactly what Flash Attention does, and what the next part formalizes as tiling and fusion.

PART II

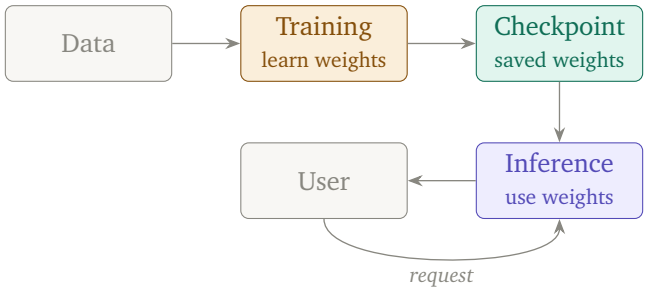
Training

Chapter 3

Training vs Inference

Book 1 deliberately stayed on the inference side. This chapter orients you: where does training fit, and why is it a different engineering problem?

3.1 Two Different Jobs



- **Training** runs once (or periodically), consumes enormous compute over days or weeks, processes a fixed dataset, and produces a checkpoint — the learned weights. It is a throughput job: maximize useful work per GPU-hour.
- **Inference** runs constantly, serves unpredictable live requests, and reads the frozen checkpoint. It is a latency-and-throughput job under a real-time constraint.

> **In plain terms** Training is writing the textbook: slow, expensive, done once by a few authors. Inference is reading the textbook: fast, repeated, done by millions. The same weights, two completely different operational problems.

3.2 Why Training Needs So Much More Memory

Inference stores weights plus the KV cache. Training stores far more, because to improve the weights it must remember how it got each answer.

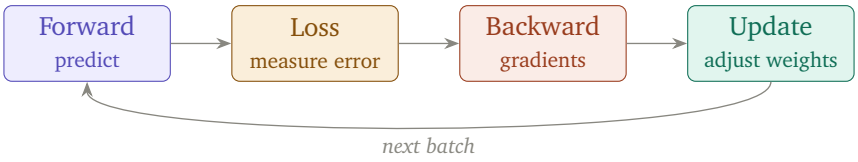
Stored in training	Purpose
Weights	the model itself
Gradients	how to change each weight (same size as weights)
Optimizer state	momentum and variance per weight (Adam: $2\times$ weights)
Activations	every intermediate value, kept for the backward pass

A rule of thumb: training a model needs roughly 4–$8\times$ the memory of merely holding its weights, once gradients, Adam optimizer state, and activations are counted. This is why a model you can *infer* on one GPU may need a cluster to *train*.

Chapter 4

The Training Loop and Mixed Precision

4.1 Forward, Loss, Backward, Update



Each step: run a batch forward to get predictions, compute the loss against the true next tokens, run backpropagation to get a gradient for every weight, and update the weights with the optimizer. Repeat across billions of tokens.

4.2 Mixed Precision

Training in full 32-bit precision is slow and memory-hungry. Mixed precision keeps most math in 16-bit (FP16 or BF16) to feed tensor cores fast, while keeping a 32-bit master copy of the weights for numerical stability.

- **BF16** (bfloat16) has the same exponent range as FP32 with fewer mantissa bits. It rarely overflows, so it has become the default for large-model training.
- **FP16** has more precision but a narrow range, requiring loss scaling to avoid underflow.

> Mixed precision is the training-side mirror of inference quantization: both pick the smallest numeric format that preserves quality, to move less data and run tensor cores faster. The pyramid lesson from Part I applies on both sides of the model's life.

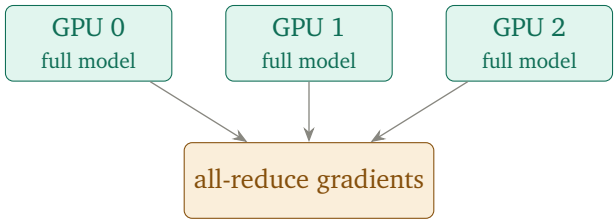
Chapter 5

Distributed Training

A frontier model does not fit on one GPU, and even if it did, training on one GPU would take years. Distributed training splits the work. There are four main strategies, often combined.

5.1 Data Parallelism

Every GPU holds a full copy of the model and processes a different slice of the batch. After each step, gradients are averaged across GPUs (an *all-reduce*) so all copies stay identical.



Simple and scalable, but every GPU must hold the whole model — useless once the model exceeds one GPU's memory.

5.2 Tensor Parallelism

Split each layer's weight matrices across GPUs; they cooperate on every token, exchanging partial results. This is the same mechanism Book 1 introduced for inference. It needs constant fast communication, so it stays within an NVLink node.

5.3 Pipeline Parallelism

Assign contiguous layer ranges to different GPUs; activations flow through the pipeline like an assembly line. Communicates less than tensor parallelism, so it can span nodes, but suffers *pipeline bubbles* — GPUs idling while waiting for the stage before them. Micro-batching reduces the bubbles.

5.4 Fully Sharded Data Parallelism (FSDP / ZeRO)

The memory problem with plain data parallelism is the redundant full copy on every GPU. FSDP shards the weights, gradients, and optimizer state across GPUs, gathering each layer's parameters only when needed, then releasing them.

3D parallelism combines all three: data parallelism across replicas, tensor parallelism within a node, pipeline parallelism across nodes, with FSDP sharding to cut memory. Training a frontier model is largely the art of choosing this combination to fit the model on the cluster while keeping every GPU busy.

5.5 MoE Training

Mixture-of-Experts (Book 1, Chapter 5) adds *expert parallelism*: experts are spread across GPUs, and routing a token means sending it to the GPU holding its chosen expert — an *all-to-all* communication pattern. The training challenge is load balancing: an auxiliary loss discourages the router from overusing a few favourite experts, which would leave most GPUs idle.

In plain terms Imagine the assembly line, the team-per-station, and the duplicate-factories ideas all running at once, plus a mail system that routes each half-built product to whichever specialist factory handles it. That combination, kept balanced so no factory is overwhelmed, is how the largest models are trained.

PART III

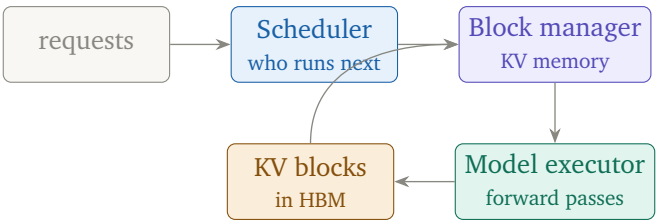
Inference Engine Internals

Chapter 6

Inside vLLM

Book 1 used vLLM as a black box that does continuous batching and PagedAttention. This chapter opens it. Understanding one engine deeply makes every other engine legible.

6.1 The Components

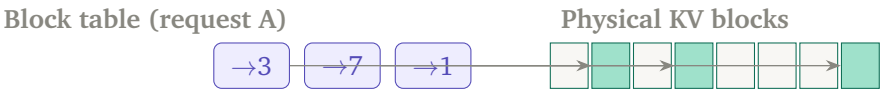


- **Scheduler**: at every step, decides which requests run, admits new ones into the running batch (continuous batching), and preempts requests if memory runs short.
- **Block manager**: allocates and frees KV-cache memory in fixed blocks — the heart of PagedAttention.
- **Model executor**: runs the actual prefill and decode forward passes on the GPU, often across multiple GPUs via tensor parallelism.

6.2 PagedAttention in Detail

Book 1 said PagedAttention treats KV memory like virtual-memory pages. Here is the mechanism. Classic serving allocated one big contiguous KV region per request, sized to the maximum possible length. Two problems followed: *internal fragmentation* (a request that ends early wastes its reserved tail) and *external fragmentation* (free gaps too small to reuse).

PagedAttention breaks the KV cache into fixed-size **blocks** (say 16 tokens each) and keeps a per-request **block table** mapping logical positions to physical blocks — exactly like an operating system's page table.



Because blocks need not be contiguous, there is almost no wasted memory, and two requests sharing a prefix can point their block tables at the *same* physical blocks — this is how prefix caching is implemented.

> PagedAttention is the single change that let vLLM serve far more concurrent requests than prior engines: by eliminating KV fragmentation, the same GPU memory holds many more sequences. It is a systems idea (OS paging) imported into ML serving.

6.3 RadixAttention and SGLang

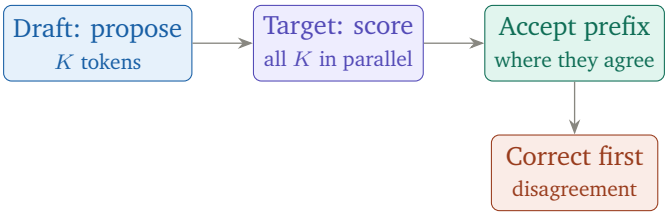
SGLang generalizes prefix sharing with **RadixAttention**: instead of only sharing a single common prefix, it stores all cached prefixes in a radix tree, so *any* shared prefix among any set of requests is reused automatically. For workloads with branching shared context — few-shot prompts, tree-of-thought, multi-turn agents — this reuses far more of the KV cache than linear prefix caching.

Chapter 7

Speculative Decoding and Quantization Internals

7.1 How Speculative Decoding Is Implemented

Book 1 gave the idea: a draft model proposes, the target verifies. The implementation detail that makes it correct is *rejection sampling*.



The target model processes all K draft tokens in a single forward pass (like a mini-prefill, high arithmetic intensity). It accepts the longest prefix consistent with its own distribution and resamples at the first disagreement. Crucially, the accept/reject rule is designed so the output distribution is *identical* to running the target model alone — speculative decoding is a pure speedup, not an approximation.

> The speedup depends entirely on the *acceptance rate*. A draft model too weak gets rejected often (little gain, wasted draft compute); too strong and it is as slow as the target. The art is a draft model that is cheap yet agrees with the target most of the time — often a small model from the same family, or the target's own early layers.

7.2 Quantization Internals

Book 1 listed INT8 and INT4 as memory savings. Here is what actually happens to the numbers.

- **Post-training quantization (PTQ)**: take a trained FP16 model and convert weights to low precision afterward. Fast, no retraining. Methods like GPTQ and AWQ choose the rounding carefully to minimize error on important weights.
- **Quantization-aware training (QAT)**: simulate low precision during training so the model learns to tolerate it. More accurate at very low bit-widths, far more expensive.

The core operation is mapping a continuous range of FP16 values onto a small grid of integers

via a *scale* (and sometimes a *zero point*):

$$q = \text{round}\left(\frac{x}{\text{scale}}\right), \qquad x \approx q \times \text{scale}$$

> The reason INT4 can degrade reasoning while barely touching simple tasks: **outlier weights**. A few large-magnitude values carry disproportionate information, and a coarse 4-bit grid rounds them badly. AWQ and similar methods detect these salient weights and protect them with finer scales — which is why *how* you quantize matters as much as the bit-width.

7.3 KV-Cache Compression

The KV cache, not the weights, is often the memory bottleneck at long context (Book 1, Part II). Beyond GQA, three families compress it further:

Approach	Idea
KV quantization	store K and V in INT8/INT4 instead of FP16
Token eviction	drop cache entries for tokens unlikely to be attended again
Low-rank / merging	compress or merge KV entries that carry similar information

Each trades a little accuracy for a longer effective context or more concurrent requests — the same memory-versus-quality dial that runs through both books.

PART IV

Kernels

Chapter 8

Writing a GPU Kernel

This is where systems engineering meets the metal. You will not write production kernels on day one, but reading them and reasoning about their speed is a core skill. We build intuition with the canonical example: matrix multiplication.

8.1 The Naive Kernel

The simplest matmul kernel assigns one output element to one thread. Each thread reads a full row and a full column from global memory and computes their dot product.

```
__global__ void matmul_naive(float* C, float* A,
    float* B, int N) {
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;
  float acc = 0;
  for (int k = 0; k < N; k++)
    acc += A[row*N + k] * B[k*N + col];
  C[row*N + col] = acc;
}
```

This is correct but slow. Every thread reads its row and column straight from HBM, and neighbouring threads re-read the same data over and over. The kernel is memory-bound — it spends its time waiting on global memory, exactly the failure mode Part I warned about.

8.2 Tiling: The Key Idea

A **tiled** kernel loads small square tiles of A and B into shared memory once, and every thread in the block reuses them. Data travels from slow HBM to fast SRAM a single time, then is read many times from SRAM.

In plain terms The naive kernel is like every student walking to the library for each single fact. The tiled kernel sends one student to fetch a stack of books to the shared desk (SRAM); then the whole team reads from the desk. Far fewer trips to the slow library (HBM).

> Tiling is the most important kernel-optimization pattern, and it is the foundation of Flash Attention. The speedup comes not from doing less arithmetic — the FLOP count is identical — but from drastically reducing how many times data crosses the slow HBM boundary. Arithmetic intensity goes up; the kernel moves up the roofline.

8.3 Fusion

A second pattern: **kernel fusion** combines several operations into one kernel so intermediate results never leave SRAM. Instead of (matmul → write to HBM → read from HBM → add bias → write → read → activation), a fused kernel does matmul-bias-activation in one pass, writing to HBM only once.

Chapter 9

Flash Attention From First Principles

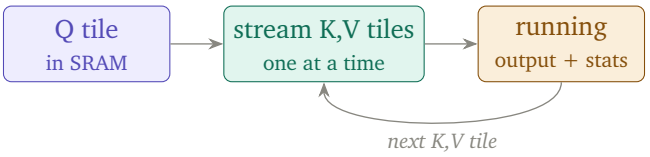
Now we can fully explain the optimization Book 1 named repeatedly. Flash Attention is tiling and fusion applied to the attention operation, plus one clever trick for the softmax.

9.1 The Problem It Solves

Standard attention computes the full $N \times N$ score matrix, writes it to HBM, reads it back to apply softmax, reads it again to multiply by V. For long sequences this matrix is huge and the HBM traffic dominates — attention becomes memory-bound and the N^2 memory cost limits context length.

9.2 The Tiling

Flash Attention never materializes the full score matrix. It walks over the keys and values in tiles, loading each tile into SRAM, computing the partial scores against the query tile, and immediately folding them into a running output — all without writing the score matrix to HBM.



9.3 The Online Softmax Trick

The obstacle is that softmax needs the maximum and the sum over *all* scores to normalize — but Flash Attention only sees one tile at a time. The solution is the **online softmax**: keep a running maximum and a running sum, and rescale the accumulated output each time a new tile reveals a larger maximum. After the last tile, the result is exactly the true softmax, computed without ever holding all scores at once.

> This is the whole trick: by combining tiling (keep data in SRAM), fusion (never write the score matrix), and online softmax (normalize incrementally), Flash Attention turns attention from $O(N^2)$ HBM traffic into $O(N)$. The arithmetic is unchanged; the memory movement collapses. Every concept in this book converges here — the memory hierarchy, tiling, fusion, and the roofline all explain one kernel.

This is why Flash Attention is described as “IO-aware” rather than “fewer FLOPs.” It does the same number of multiplications as standard attention. Its entire advantage is moving less data across the slow boundary — a systems optimization, not a mathematical one.

PART V

Operating LLM Systems

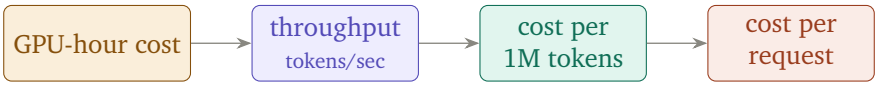
Chapter 10

Cost Modeling

Book 1 measured latency and throughput. Production systems must also answer: what does this cost? For an ML systems engineer, turning performance numbers into dollars is a core skill.

10.1 From GPU-Hours to Dollars

The base unit is the GPU-hour — the rental or amortized cost of one GPU for one hour. Everything else derives from it and from throughput.



The chain:

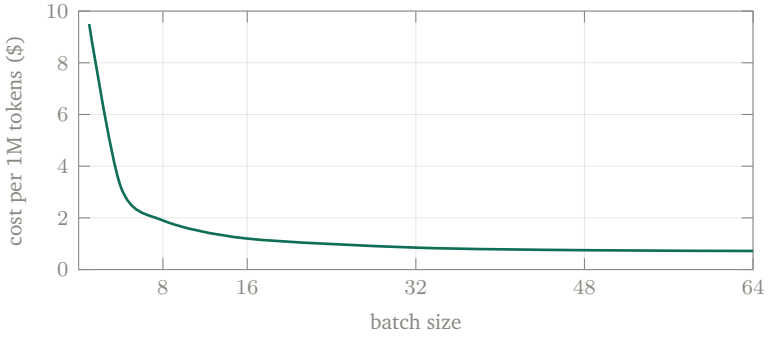
$$\text{cost per token} = \frac{\text{GPU-hour cost}}{3600 \times \text{tokens per second}}$$

$$\text{cost per request} = \text{cost per token} \times (\text{input} + \text{output tokens})$$

In plain terms You rent a GPU by the hour. If it produces 2,000 tokens per second, in an hour it makes 7.2 million tokens. Divide the hourly rent by 7.2 million and you have the cost of one token. Multiply by how many tokens a request uses, and you have the cost of that request. Everything else is detail.

10.2 Why Batching Dominates Unit Cost

A single request leaves most of the GPU idle (decode is memory-bound, Book 1 Part II). Batching many requests through the same forward passes spreads the GPU-hour across far more tokens, so cost per token falls sharply with batch size — until memory or latency limits cap the batch.



This curve is why serving economics reward high utilization. The same model on the same GPU can cost an order of magnitude more per token at batch 1 than at batch 32. Capacity planning is largely the search for the batch size that minimizes cost without violating your latency budget.

10.3 The Levers

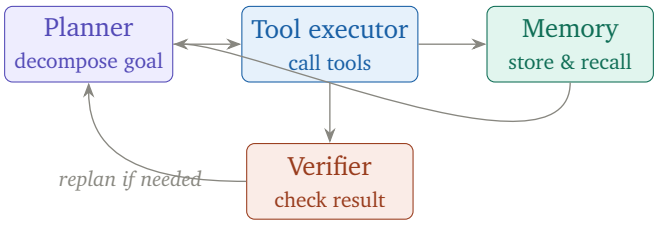
Lever	Effect on cost
Quantization	smaller weights/KV ⇒ bigger batches fit ⇒ lower cost per token
Prefix caching	shared prefills become free ⇒ lower cost on shared-context workloads
Speculative decoding	more tokens per GPU-second ⇒ lower cost when acceptance is high
Right-sizing the GPU	an L40S may beat an H100 on $/token for small models

Chapter 11

Agent System Architecture

Book 1's agent chapter focused on context engineering — managing the evolving state. This chapter gives the surrounding architecture: the components that turn a model into an agent.

11.1 The Core Loop



- **Planner**: breaks a goal into steps, decides the next action. Often the LLM itself, prompted to reason about what to do next.
- **Tool executor**: runs the chosen tool (search, code, API call) and returns the result into context, shaped to only what matters (Book 1's tool-output shaping).
- **Memory**: short-term (the working context) and long-term (durable facts in a store, often a vector DB), summarized and pruned to fit the window.
- **Verifier**: checks whether a step succeeded — a test suite, a rule, or another model call — and triggers a retry or replan on failure.

In plain terms An agent is the reasoning model from Book 1 placed inside a loop with hands and a notebook. The planner decides what to do, the executor does it with tools, the verifier checks the work, and the memory keeps track. The loop repeats until the goal is met or abandoned.

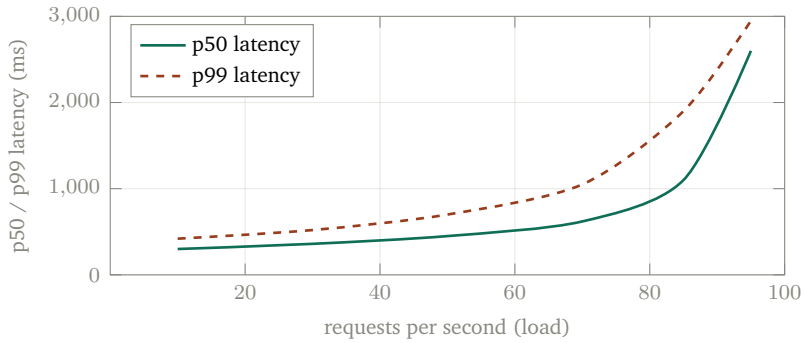
Every box here is a multiplier on inference cost. A single agent task may issue dozens of LLM calls — plan, act, verify, replan — each a full request with its own prefill and decode. This is why the cost modeling of the previous chapter matters most for agents: the unit is no longer a request but an entire multi-step trajectory.

Chapter 12

Serving Benchmarks and Capacity Planning

12.1 What to Measure

Benchmarking a serving system is not one number. The honest report is a curve: how latency behaves as load rises.



The system is healthy in the flat region and falls over at the *knee*, where latency spikes as the queue saturates. Capacity planning means operating below the knee with headroom.

> Report **percentiles**, never just the average. A mean latency of 400 ms can hide a p99 of 3 seconds — meaning one user in a hundred has a terrible experience. Under load, the tail (p95, p99) degrades long before the median, and the tail is what users remember.

12.2 The Metrics That Matter

Metric	Why it matters
TTFT (p50, p99)	perceived responsiveness, including queue delay
Inter-token latency	smoothness of streaming after the first token
Throughput at SLA	max requests/sec while still meeting the latency budget
Goodput	throughput of requests that actually met their deadline
Cost per 1M tokens	the economics from the cost chapter

> Capacity planning ties the whole two-book arc together: you size a deployment by finding the batch size and replica count that maximize throughput-at-SLA and minimize cost per token, given the model's memory footprint and your latency budget. Every concept — KV cache size, quantization, batching, parallelism — feeds into that single sizing decision.

PART VI

Hands-On Labs

Chapter 13

Systems Labs

Book 1's labs measured the application layer. These six go beneath it — profiling hardware, writing a kernel, running distributed, modeling cost, and benchmarking a server. They assume the same lightweight setup plus a profiler.

> **In plain terms** A few of these (the kernel and distributed labs) want a real NVIDIA GPU. The cost-model and benchmark labs run anywhere. Do what your hardware allows; even reading the expected results with the diagrams from earlier chapters builds the mental model.

Setup

```
# Profiling and kernel tools
pip install nvidia-ml-py torch
# NVIDIA Nsight Systems / Compute for kernel profiling
# (nsys, ncu) ship with the CUDA toolkit
```

Chapter 14

Lab 1 — Profile the Memory Hierarchy

Reinforces: Part I (GPU architecture, memory hierarchy).

Hypothesis: a decode-heavy workload is memory-bound; the GPU's compute units sit mostly idle.

```
# Profile a generation run and read the roofline:
ncu -set full -o decode_profile \
  python generate.py
# Inspect achieved occupancy, DRAM throughput,
# and compute throughput in the report.
```

Expected: DRAM (HBM) throughput near its ceiling while compute throughput is low. *Why:* decode reads the KV cache and weights from HBM with little arithmetic per byte — it is pinned to the memory diagonal of the roofline.

Chapter 15

Lab 2 — Tiled vs Naive Matmul

Reinforces: Part IV (kernels, tiling).

Hypothesis: a tiled kernel beats a naive one with identical FLOPs, purely by reducing HBM traffic.

```
# Implement both kernels (see Chapter 8),
# launch on the same NxN matrices, and time them.
# Then profile HBM bytes moved for each:
ncu -metrics dram__bytes.sum naive tiled
```

Expected: the tiled kernel is several times faster and moves far fewer HBM bytes, despite doing the same multiplications. *Why:* tiling reuses data from SRAM, raising arithmetic intensity.

Chapter 16

Lab 3 — Run Data-Parallel Training

Reinforces: Part II (distributed training).

Hypothesis: data parallelism scales throughput nearly linearly until communication overhead bites.

```
torchrun -nproc_per_node=N train.py \
  -strategy ddp
# Record tokens/sec for N = 1, 2, 4 GPUs.
# Then repeat with FSDP and compare peak memory.
```

Expected: near-linear throughput scaling at small N; FSDP shows lower peak memory per GPU than plain DDP. *Why:* DDP replicates the model (fast but memory-heavy); FSDP shards it (memory-light, more communication).

Chapter 17

Lab 4 — Build a Cost Model

Reinforces: Part V (cost modeling).

Hypothesis: cost per token falls steeply with batch size, then flattens.

```
def cost_per_million(gpu_hourly, tokens_per_sec):
  tokens_per_hour = tokens_per_sec * 3600
  return gpu_hourly / tokens_per_hour * 1_000_000
# Sweep batch size, record tokens/sec from vLLM,
# plot cost_per_million against batch size.
```

Expected: the curve from Chapter 10 — order-of-magnitude cheaper at batch 32 than batch 1, then diminishing returns. *Why:* batching amortizes the GPU-hour across far more tokens until memory or latency caps the batch.

Chapter 18

Lab 5 — Benchmark a Server Under Load

Reinforces: Part V (serving benchmarks, capacity planning).

Hypothesis: latency stays flat until a knee, then spikes; the tail degrades before the median.

```
# Drive increasing load and record percentiles:
python -m vllm.entrypoints.openai.api_server ...  &
# Use a load tool (e.g.  a concurrency sweep) to
# push 10, 30, 50, 70, 90 req/s; log p50 and p99.
```

Expected: the load-vs-latency curve from Chapter 12, with p99 rising well before p50. *Why:* as the queue saturates, tail requests wait behind others; the knee marks your safe capacity.

Chapter 19

Lab 6 — Measure Speculative Decoding

Reinforces: Part III (speculative decoding internals).

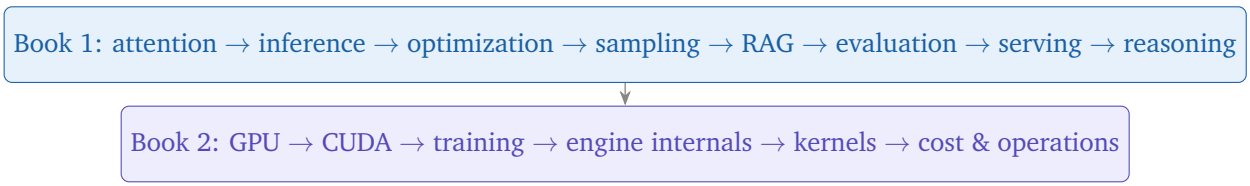
Hypothesis: speedup tracks the acceptance rate, which depends on draft-model quality.

```
# Enable speculative decoding in vLLM with a small
# draft model; log acceptance rate and tokens/sec.
# Repeat with a weaker and a stronger draft model.
```

Expected: a stronger draft model raises acceptance and tokens/sec, up to the point where the draft itself is too slow. *Why:* accepted tokens come nearly free in the target's parallel verification; rejected tokens waste draft compute.

Closing: The Two-Book Arc

Book 1 built understanding from the token down to the served request. Book 2 went beneath that — from the served request down to the silicon, the kernels, and the cluster. Read together they form one continuous explanation:



The recurring lesson across both volumes is a single sentence: *the binding constraint is almost never raw compute — it is memory, bandwidth, and communication.* Attention caching, Flash Attention, quantization, PagedAttention, GQA, MoE, FSDP, and tiling are all the same move at different scales: keep data close, move it less, and keep every processor fed.

An engineer who can hold this whole arc in their head — and has run the labs to feel it — can not only use LLM systems but design and optimize them.

Further Reading

- **CUDA C++ Programming Guide:** https://docs.nvidia.com/cuda/cuda-c-programming-guide/
- **NVIDIA Nsight Compute:** https://docs.nvidia.com/nsight-compute/
- **Flash Attention:** https://github.com/Dao-AILab/flash-attention
- **vLLM:** https://docs.vllm.ai
- **SGLang / RadixAttention:** https://github.com/sgl-project/sglang
- **PyTorch FSDP:** https://pytorch.org/docs/stable/fsdp.html
- **Megatron-LM (3D parallelism):** https://github.com/NVIDIA/Megatron-LM
- **TensorRT-LLM:** https://nvidia.github.io/TensorRT-LLM/